Name - S. Jaxon Rizal
Reg. No - 192511421
Course Code - CSA0313

CO5 - AT1 - PSEUDOCODE WRITING TASK (Any 10)

1. Write pseudocode for Breadth First Search (BFS) Traversal.

```
Algorithm BFS (G, start)
1. for each vertex v in G
       visited[v] ← false
2. create empty queue Q
3. visited[start] ← true
4. enqueue (Q, start)
5. while Q is not empty
       u ← dequeue (Q)
       visit u
       for each vertex v adjacent to u
           if visited[v] = false
               visited[v] ← true
               enqueue (Q, v)
```

2. Write pseudocode for Depth First Search (DFS) Traversal.

```
Algorithm DFS (G, v)
    1. visited [v] ← true
    2. visit v
    3. for each vertex u adjacent to v
           if visited [u] = false
               DFS (G, u)
```

3. Write pseudocode for Breadth First Search (BFS) Traversal.

```
Algorithm BFS (G, start)
    1. for each vertex v in G
           visited [v] ← false
    2. create empty queue Q
    3. visited [start] ← true
    4. enqueue (Q, start)
    5. while Q is not empty
           u ← dequeue (Q)
           visit u
           for each vertex v adjacent to u
               if visited [v] = false
                   visited [v] ← true
                   enqueue (Q, v)
```

4. Write pseudocode for Dijkstra's Shortest Path Algorithm.

```
Algorithm Dijkstra (G, source)

1. for each vertex v in G
       distance[v] ← ∞
       visited[v] ← false
2. distance[source] ← 0
3. for i ← 1 to |V|
       u ← vertex with minimum
         distance[u] not yet visited
       visited[u] ← true
       for each vertex v adjacent to u
           if visited[v] = false and
              distance[u] + weight(u, v) < distance[v]
              distance[v] ← distance[u] + weight(u, v)
4. return distance
```

5. Write pseudocode for Prim's Minimum Spanning Tree Algorithm.

Algorithm PrimMST(G)

1. for each vertex v in G
   key[v] ← ∞
   parent[v] ← NIL
2. choose any vertex r as start vertex
   key[r] ← 0
   parent[r] ← NIL
3. create a min priority queue Q
   insert all vertices of G into Q
   with key values
4. while Q is not empty
   u ← extract-min(Q)
5. for each vertex v adjacent to u
   if v is in Q and weight(u,v) < key[v]
   key[v] ← weight(u,v)
   parent[v] ← u
   decrease-key(Q, v, key[v])
6. return parent[] which represents MST

6. Write pseudocode for Floyd-Warshall Algorithm.

```
Algorithm FloydWarshall (W)
1. for i ← 1 to |V|
       for j ← 1 to |V|
           D[i][j] ← W[i][j]
2. for i ← 1 to |V|
       D[i][i] ← 0
3. for k ← 1 to |V|
       for i ← 1 to |V|
           for j ← 1 to |V|
               if D[i][k] + D[k][j] < D[i][j]
                   D[i][j] ← D[i][k] + D[k][j]
4. return D
```

7. Write pseudocode for Prim's Minimum Spanning Tree Algorithm

```
Algorithm PrimMST (G, start)
1. for each vertex v in G
       key[v] ← ∞, parent[v] ← NIL
2. key[start] ← 0
3. create min priority queue Q and insert all vertices
4. while Q is not empty
       u ← extract-min(Q)
       for each vertex v adjacent to u
           if v is in Q and weight(u,v) < key[v]
               key[v] ← weight(u,v)
               parent[v] ← u
5. return parent
```

8. Write pseudocode for Kruskal's Minimum Spanning Tree Algorithm.

```
Algorithm KruskalMST ( G , start )
    1. for each vertex v in G
           make-set(v)
    2. sort all edges of G in non-decreasing order of weight
    3. MST ← ∅
    4. for each edge (u, v) in sorted order
           if find-set(u) ≠ find-set(v)
               MST ← MST ∪ {(u, v)}
               union (u, v)
    5. return MST
```

9. Write pseudocode for Topological Sort using Kahn's Algorithm.

```
Algorithm TopologicalSort ( G )
    1. for each vertex v in G
           indegree[v] ← 0
    2. for each edge ( u , v ) in G
           indegree[v] ← indegree[v] + 1
    3. create empty queue Q
    4. for each vertex v in G
           if indegree[v] = 0
               enqueue ( Q , v )
    5. create empty list L
    6. while Q is not empty
           u ← dequeue(Q)
           append u to L
           for each vertex v adjacent to u
               indegree[v] ← indegree[v] - 1
               if indegree[v] = 0
                   enqueue ( Q , v )
    7. if length of L = |V|
           return L ( topological order)
       else
           return "Graph has a cycle, topological sort not possible"
```

10. Write pseudocode for Dijkstra's Shortest Path Algorithm.

```
Algorithm Dijkstra ( G , source)

1. for each vertex v in G
       distance[v] ← ∞
       visited[v] ← false
       parent[v] ← NIL

2. distance[source] ← 0

3. create min priority queue Q and insert all vertices
       with key as distance[v]

4. while Q is not empty
       u ← extract-min ( Q )
       visited[u] ← true
       for each vertex v adjacent to u
           if visited[v] = false
               alt ← distance[u] + weight(u,v)
               if alt < distance[v]
                   distance[v] ← alt
                   parent[v] ← u
                   decrease-key ( Q , v , alt )

5. return distance , parent
```